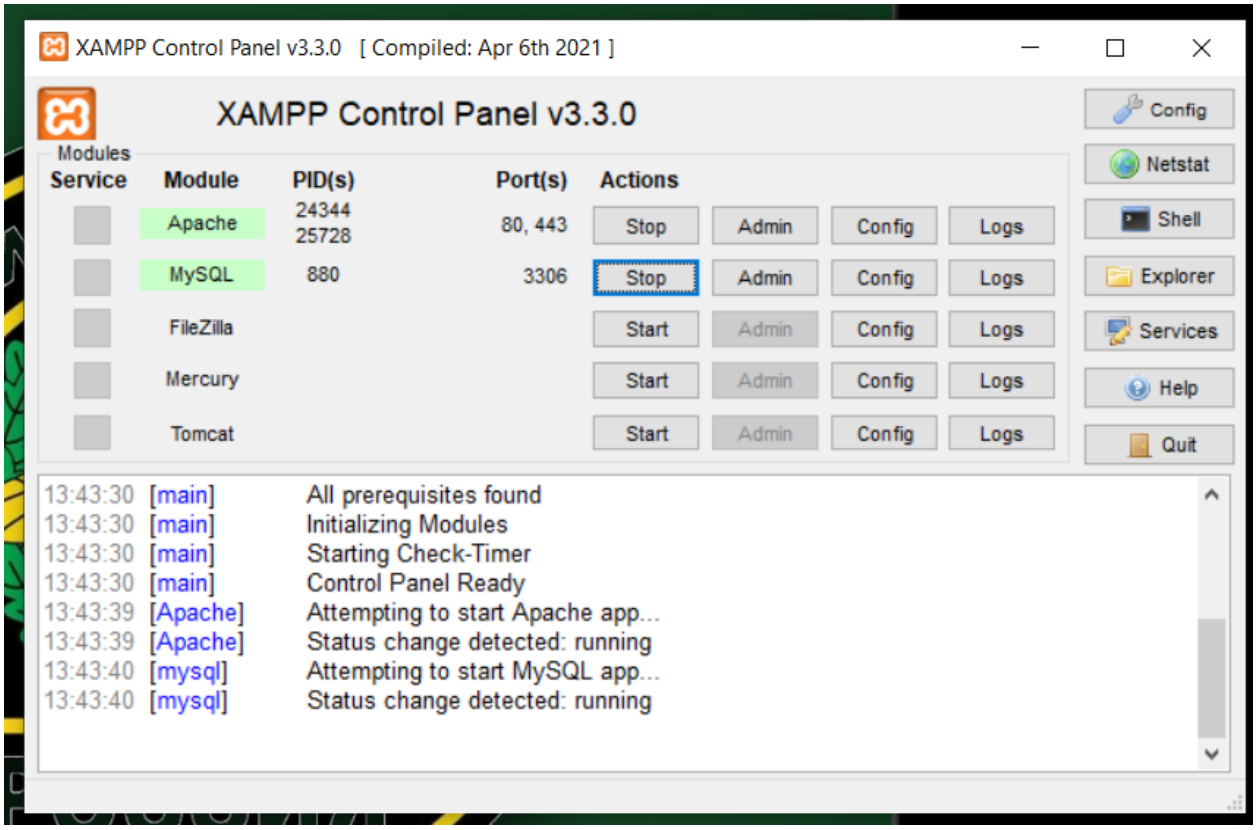
Задание 1. Разработка системы хранения данных

Предметная область (упрощённо)

Преподаватель ведёт несколько дисциплин. Студенты учатся в группах. Преподаватель выставляет/изменяет оценки студентам по своим дисциплинам. Студент видит только свои оценки. При «удалении» оценки данные не теряются, а помечаются как удалённые (мягкое удаление / архивирование).

Шаг 1. Запуск XAMPP

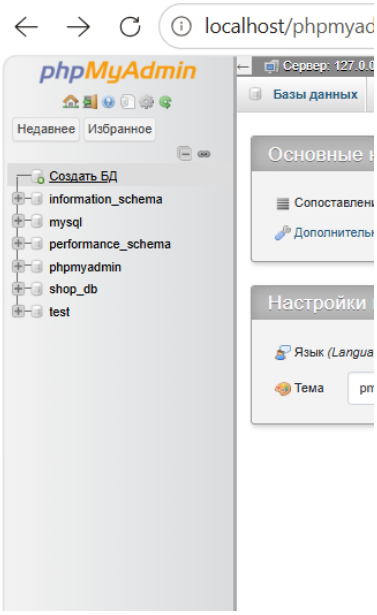
1. Откройте XAMPP Control Panel.
2. Нажмите Start напротив Apache.
3. Нажмите Start напротив MySQL.
4. Убедитесь, что оба модуля горят зелёным.



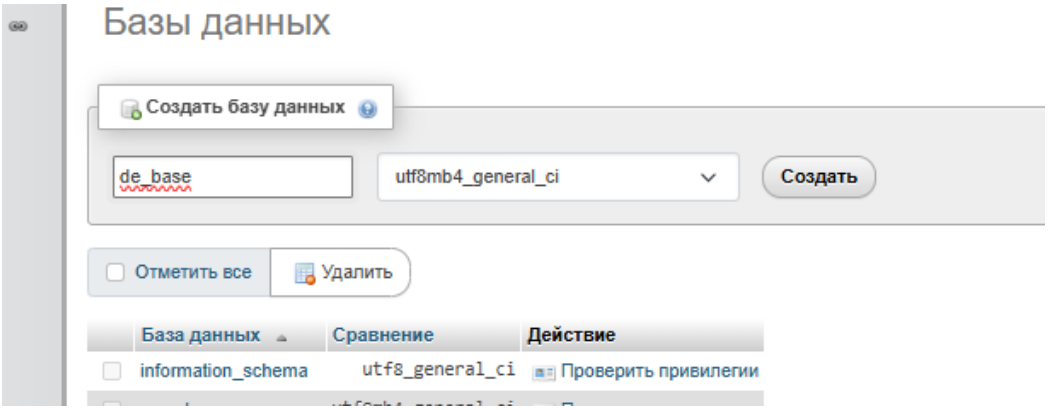
Важно: если Apache не запускается, скорее всего порт 80 занят (например, Skype или IIS). Тогда в Config → httpd.conf меняют Listen 80 на Listen 8080, и проект открывается по http://localhost:8080/....

Шаг 2. Создание базы данных в phpMyAdmin

1. В браузере откройте: http://localhost/phpmyadmin.
2. Слева нажмите «Создать БД» (или New).



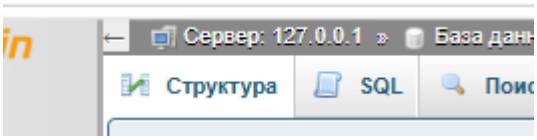
3. Введите имя базы согласно заданию)-.
4. Сравнение строк (collation): utf8mb4_general_ci.

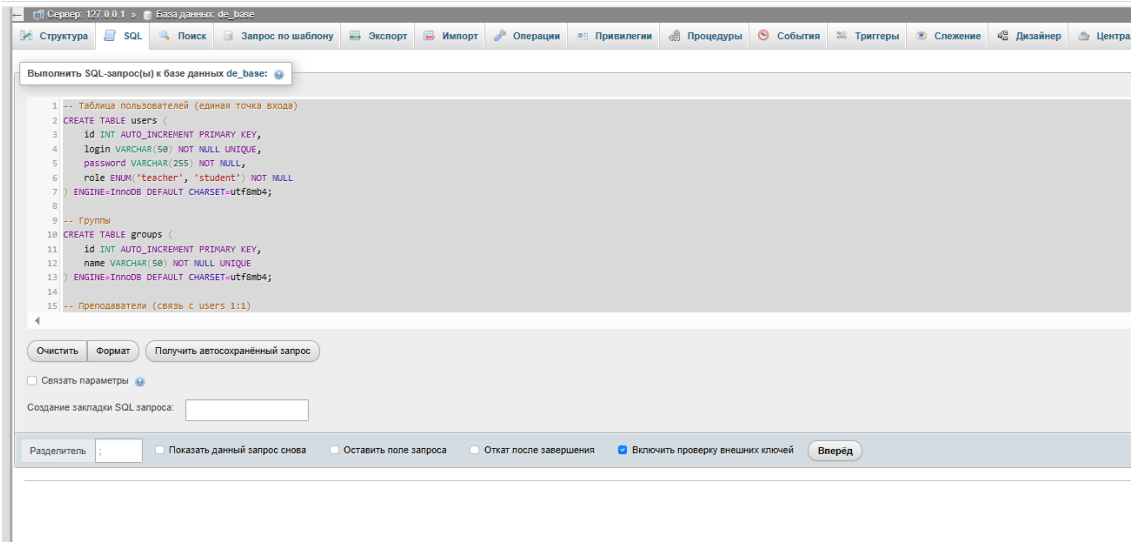


5. Нажмите «Создать».

Шаг 3. Проектирование таблиц (SQL-скрипт)

Откройте в phpMyAdmin вкладку SQL для созданной базы и выполните скрипт ниже. На экзамене этот же скрипт можно выполнить через консоль MySQL или другой клиент, если phpMyAdmin не будет.





(СПРАВА СНИЗУ НАЖАТЬ КНОПКУ ВПЕРЕД – НУЖНА ДЛЯ ЗАПУСКА КОДА)

```
-- Таблица пользователей (единая точка входа)
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    login VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role ENUM('teacher', 'student') NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Группы
CREATE TABLE groups (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Преподаватели (связь с users 1:1)
CREATE TABLE teachers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    full_name VARCHAR(150) NOT NULL,
```

```
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Студенты (связь с users 1:1 и с groups)
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    full_name VARCHAR(150) NOT NULL,
    group_id INT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (group_id) REFERENCES groups(id) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Дисциплины
CREATE TABLE disciplines (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Какие дисциплины ведёт какой преподаватель (многие-ко-многим)
CREATE TABLE teacher_disciplines (
    teacher_id INT NOT NULL,
    discipline_id INT NOT NULL,
    PRIMARY KEY (teacher_id, discipline_id),
    FOREIGN KEY (teacher_id) REFERENCES teachers(id) ON DELETE CASCADE ON UPDATE CASCADE,
```

```
    FOREIGN KEY (discipline_id) REFERENCES disciplines(id) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- Оценки
CREATE TABLE grades (
    id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    discipline_id INT NOT NULL,
    teacher_id INT NOT NULL,
    grade INT NULL, -- 2..5 или NULL, если оценки пока нет
    deleted_at DATETIME NULL DEFAULT NULL, -- NULL = активна; дата = архивирована
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (discipline_id) REFERENCES disciplines(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (teacher_id) REFERENCES teachers(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    UNIQUE KEY unique_active_grade (student_id, discipline_id, teacher_id, deleted_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Пояснения для методички:

1) ON DELETE RESTRICT на оценках и студентах не даёт случайно удалить студента, у которого уже есть оценки. Это требование «данные не должны теряться».

2) deleted_at — ключевое поле для мягкого удаления. Пока оно NULL, запись считается активной. При «удалении» мы не делаем DELETE, а ставим deleted_at = NOW().

3) UNIQUE KEY unique_active_grade не даёт создать две активные оценки по одной и той же дисциплине у одного студента от одного преподавателя. Если оценку «удалили» (поставили deleted_at), можно создать новую активную запись.

4) Пароль в учебном проекте можно хранить в открытом виде, но в методичке студентам стоит сказать: «На экзамене, если успеете, используйте password_hash() и password_verify() — это будет плюсом».

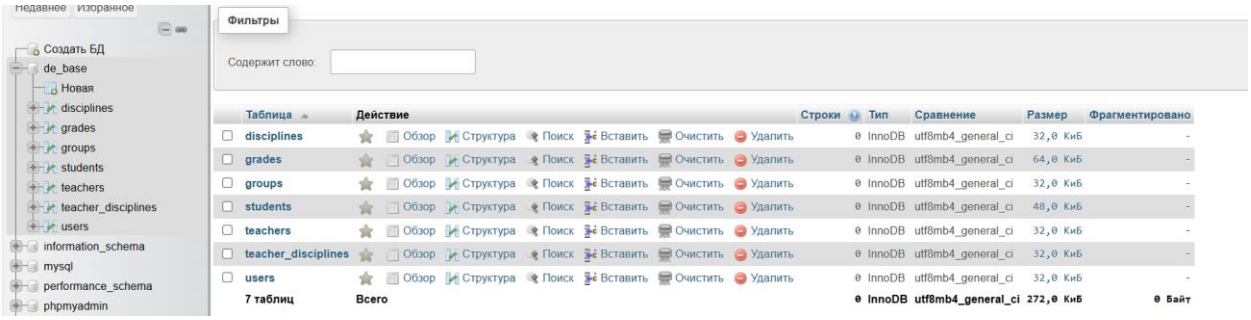


Рисунок 1 - структура таблиц

Шаг 4. Наполнение тестовыми данными (не менее 3 записей в каждой таблице)

Выполните в phpMyAdmin во вкладке SQL:

```
INSERT INTO users (login, password, role) VALUES
('teacher1', '12345', 'teacher'),
('teacher2', '12345', 'teacher'),
('student1', '12345', 'student'),
('student2', '12345', 'student'),
('student3', '12345', 'student');


INSERT INTO groups (name) VALUES
('ИС-23-1'),
('ИС-23-2'),
('ИС-22-1');


INSERT INTO teachers (user_id, full_name) VALUES
```

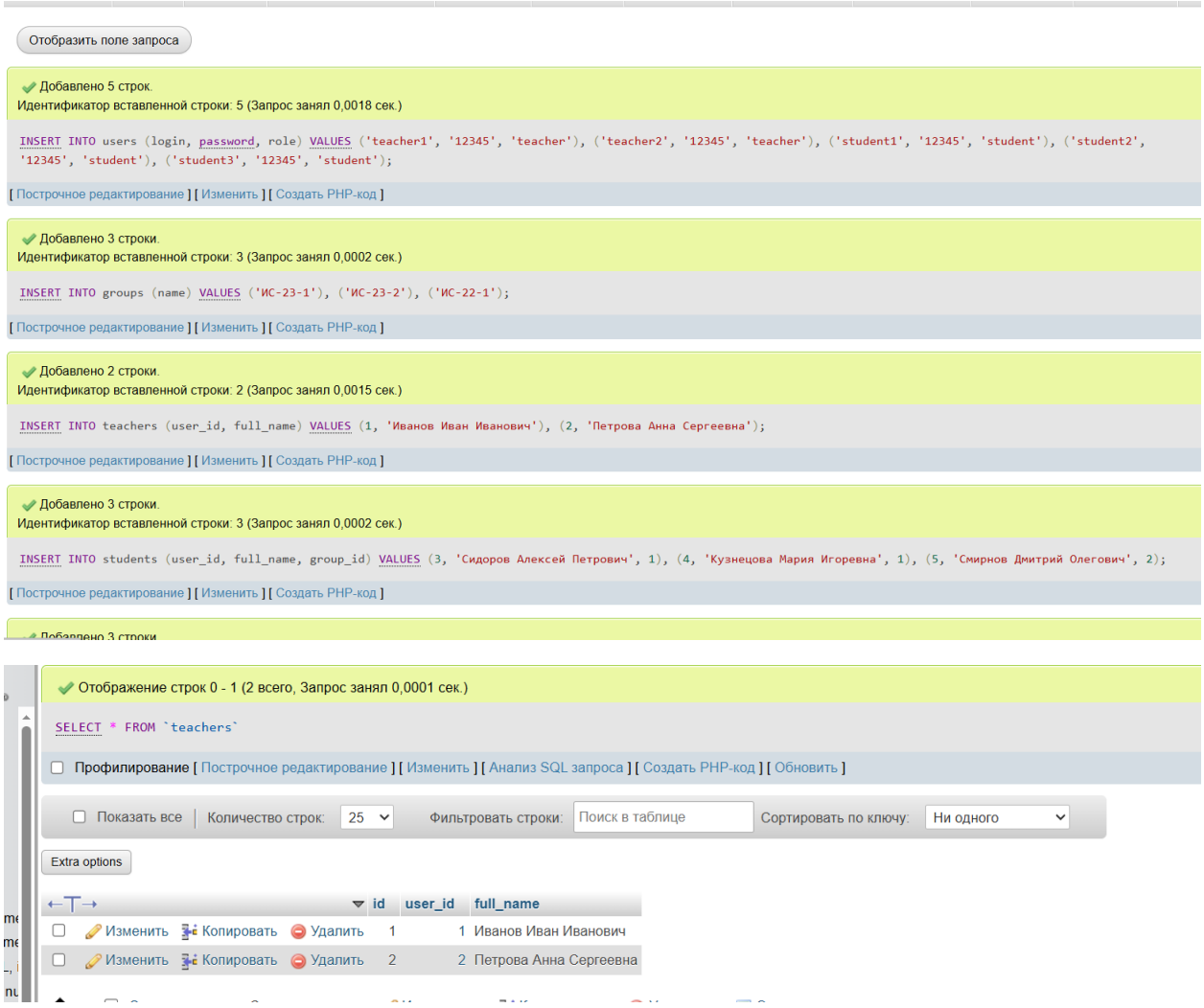
```
(1, 'Иванов Иван Иванович'),
(2, 'Петрова Анна Сергеевна');

INSERT INTO students (user_id, full_name, group_id) VALUES
(3, 'Сидоров Алексей Петрович', 1),
(4, 'Кузнецова Мария Игоревна', 1),
(5, 'Смирнов Дмитрий Олегович', 2);

INSERT INTO disciplines (name) VALUES
('Базы данных'),
('Программирование на PHP'),
('Веб-дизайн');

INSERT INTO teacher_disciplines (teacher_id, discipline_id) VALUES
(1, 1), -- Иванов ведёт Базы данных
(1, 2), -- Иванов ведёт PHP
(2, 3); -- Петрова ведёт Веб-дизайн

INSERT INTO grades (student_id, discipline_id, teacher_id, grade) VALUES
(1, 1, 1, 5),
(2, 1, 1, 4),
(1, 2, 1, 4),
(3, 3, 2, 5);
```



Шаг 5. Структура проекта в XAMPP

В проводнике перейдите в C:\xampp\htdocs\. Создайте папку, например de_grades. Внутри создайте файлы:

text

de_grades/

├── db.php # подключение к БД

├── functions.php # вспомогательные функции (проверка авторизации, экранирование)

├── login.php # форма входа

├── auth.php # обработка входа

├── logout.php # выход

├── teacher.php # главная страница преподавателя

├── save_grade.php   # обработка выставления/изменения/удаления оценки

├── student.php   # страница студента

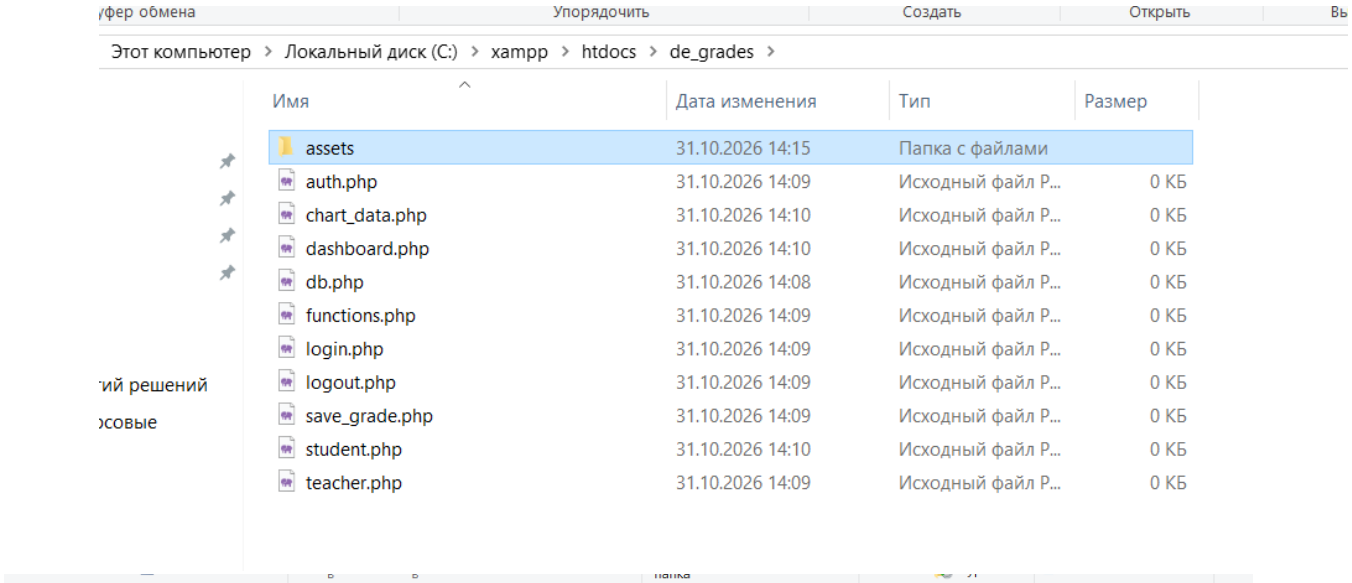
├── dashboard.php   # дашборд (задание 3)

├── chart_data.php   # выдача данных для графиков в JSON

└── assets/

    └── chart.min.js   # локальная копия Chart.js (скачать заранее!)

Важно: на экзамене Интернет может быть запрещён. Поэтому chart.min.js нужно скачать заранее (с сайта chartjs.org или из CDN) и положить в папку assets, а в HTML подключать локально: <script src="assets/chart.min.js"></script>. Если на экзамене разрешат CDN — можно использовать <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>.



Шаг 6. Подключение к базе данных (db.php)

php

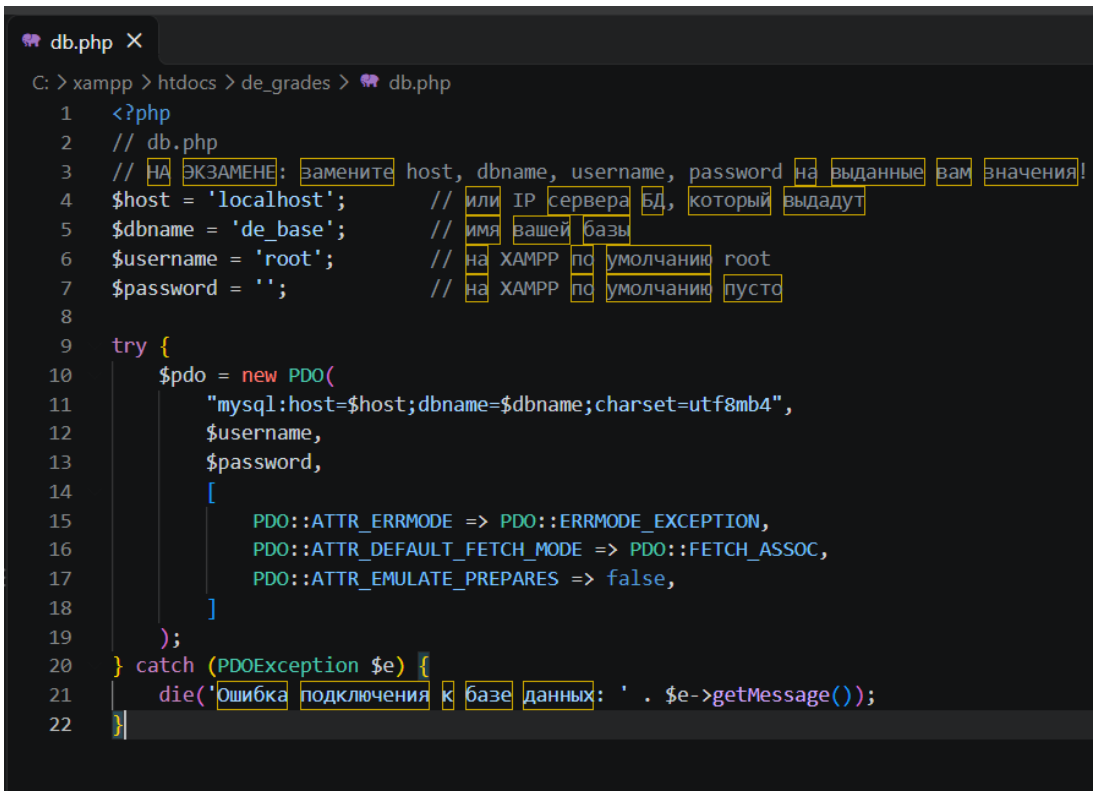
<?php

// db.php

// НА ЭКЗАМЕНЕ: замените host, dbname, username, password на выданные вам значения!

```
$host = 'localhost';       // или IP сервера БД, который выдадут
$dbname = 'de_base';       // имя вашей базы
$username = 'root';        // на XAMPP по умолчанию root
$password = '';            // на XAMPP по умолчанию пусто

try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$dbname;charset=utf8mb4",
        $username,
        $password,
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE =>
PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
        ]
    );
} catch (PDOException $e) {
    die('Ошибка подключения к базе данных: ' . $e->getMessage());
}
```



«Если на экзамене дают отдельного пользователя БД, например user_de_12 с паролем Pass123!, то меняете только эти четыре переменные. Ничего больше в коде трогать не нужно».

Шаг 7. Авторизация (login.php, auth.php, logout.php)

functions.php — вспомогательные функции:

```
php
<?php
// functions.php
session_start();


function require_login() {
    if (!isset($_SESSION['user_id'])) {
        header('Location: login.php');
        exit;
    }
}
```

```
function require_role($role) {
    require_login();
    if ($_SESSION['role'] !== $role) {
        header('Location: login.php');
        exit;
    }
}

function e($str) {
    return htmlspecialchars($str ?? '', ENT_QUOTES, 'UTF-8');
}
```

login.php — форма:

```
php
<?php require 'functions.php'; ?>
<!DOCTYPE html>
<html lang="ru">
<head><meta charset="utf-8"><title>Вход</title></head>
<body>
<h1>Вход в систему оценок</h1>
<?php if (!empty($_SESSION['error'])): ?>
    <p style="color:red"><?= e($_SESSION['error']) ?></p>
    <?php unset($_SESSION['error']); ?>
<?php endif; ?>
<form method="post" action="auth.php">
    <label>Логин: <input type="text" name="login"
required></label><br><br>
```

<label>Пароль: <input type="password" name="password" required></label>

<button type="submit">Войти</button>

</form>

</body>

</html>

auth.php — обработка:

php

<?php

require 'functions.php';

require 'db.php';

$login = trim($_POST['login'] ?? '');

$password = $_POST['password'] ?? '';

if ($login === '' || $password === '') {

$_SESSION['error'] = 'Заполните логин и пароль';

header('Location: login.php');

exit;

}

$stmt = $pdo->prepare("SELECT id, login, password, role FROM users WHERE login = ?");

$stmt->execute([$login]);

$user = $stmt->fetch();

// В учебном проекте пароль в открытом виде. В реальном — password_verify().

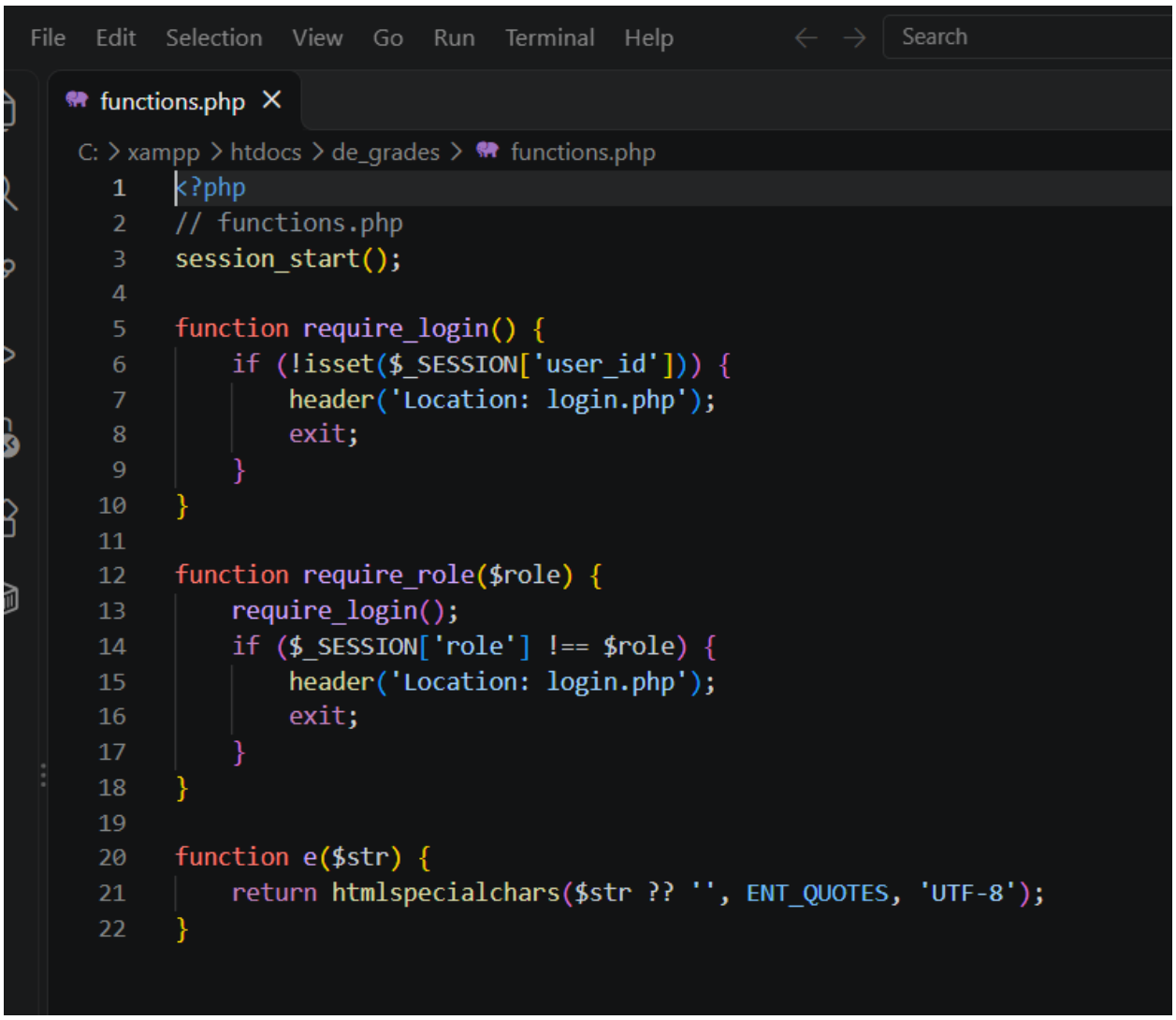
if (!$user || $user['password'] !== $password) {

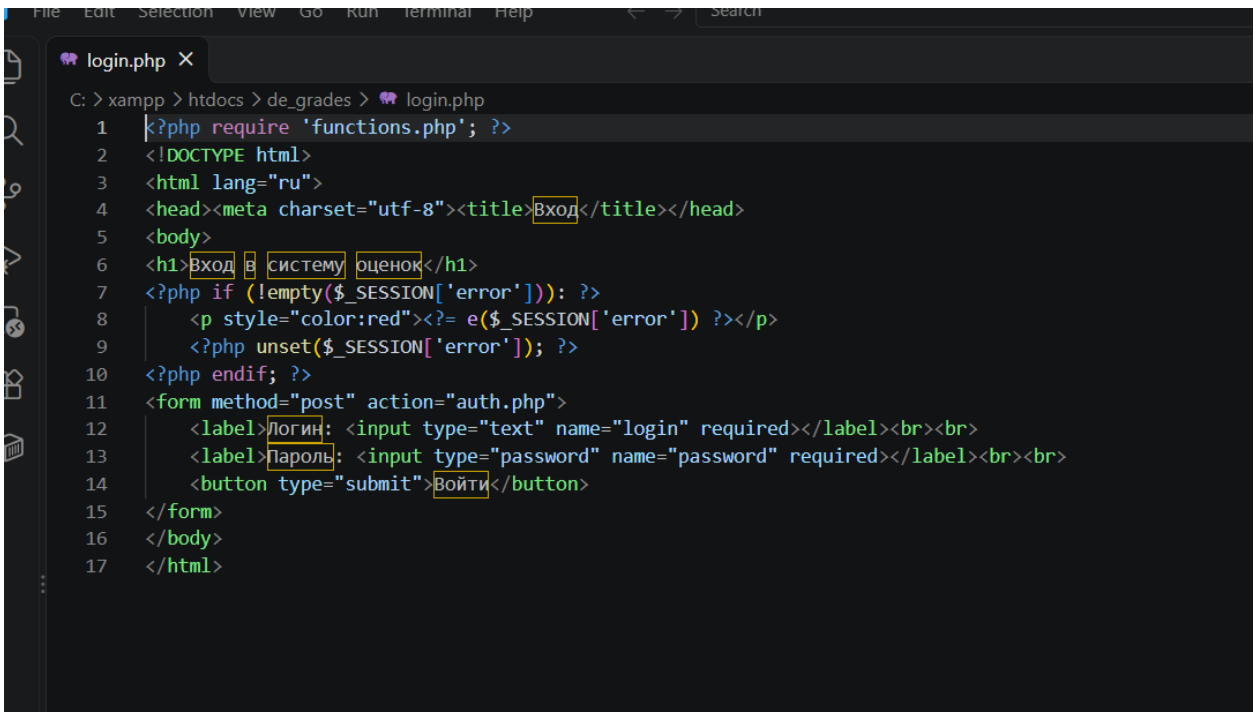
```
    $_SESSION['error'] = 'Неверный логин или пароль';
    header('Location: login.php');
    exit;
}

$_SESSION['user_id'] = $user['id'];
$_SESSION['login'] = $user['login'];
$_SESSION['role'] = $user['role'];

if ($user['role'] === 'teacher') {
    header('Location: teacher.php');
} else {
    header('Location: student.php');
}
exit;
```
logout.php:

```
php
<?php
require 'functions.php';
session_destroy();
header('Location: login.php');
exit;
```


File Edit Selection View Go Run Terminal Help
Search
functions.php
C: > xampp > htdocs > de_grades > functions.php
1 <?php
2 // functions.php
3 session_start();
4
5 function require_login() {
6 if (!isset($_SESSION['user_id'])) {
7 header('Location: login.php');
8 exit;
9 }
10 }
11
12 function require_role($role) {
13 require_login();
14 if ($_SESSION['role'] !== $role) {
15 header('Location: login.php');
16 exit;
17 }
18 }
19
20 function e($str) {
21 return htmlspecialchars($str ?? '', ENT_QUOTES, 'UTF-8');
22 }


login.php
C: > xampp > htdocs > de_grades > login.php
1 <?php require 'functions.php'; ?>
2 <!DOCTYPE html>
3 <html lang="ru">
4 <head><meta charset="utf-8"><title>Вход</title></head>
5 <body>
6 <h1>Вход в систему оценок</h1>
7 <?php if (!empty($_SESSION['error'])): ?>
8 <?= e($_SESSION['error']) ?>
9 <?php unset($_SESSION['error']); ?>
10 <?php endif; ?>
11 <form method="post" action="auth.php">
12 <label>Логин: <input type="text" name="login" required></label>

13 <label>Пароль: <input type="password" name="password" required></label>

14 <button type="submit">Войти</button>
15 </form>
16 </body>
17 </html>

File Edit Selection View Go Run Terminal Help Search

auth.php

C: > xampp > htdocs > de_grades > auth.php

```php
<?php
require 'functions.php';
require 'db.php';

$login = trim($_POST['login'] ?? '');
$password = $_POST['password'] ?? '';

if ($login === '' || $password === '') {
    $_SESSION['error'] = 'Заполните логин и пароль';
    header('Location: login.php');
    exit;
}

$stmt = $pdo->prepare("SELECT id, login, password, role FROM users WHERE login = ?");
$stmt->execute([$login]);
$user = $stmt->fetch();

// В учебном проекте пароль в открытом виде. В реальном — password_verify().
if (!$user || $user['password'] !== $password) {
    $_SESSION['error'] = 'Неверный логин или пароль';
    header('Location: login.php');
    exit;
}

$_SESSION['user_id'] = $user['id'];
$_SESSION['login'] = $user['login'];
$_SESSION['role'] = $user['role'];

if ($user['role'] === 'teacher') {
    header('Location: teacher.php');
} else {
    header('Location: student.php');
}
exit;
```

File Edit Selection View Go Run Terminal Help

logout.php

C: > xampp > htdocs > de_grades > logout.php

```php
<?php
require 'functions.php';
session_destroy();
header('Location: login.php');
exit;
```

Шаг 8. Функционал преподавателя (teacher.php и save_grade.php)

Логика:

1) Преподаватель видит свои дисциплины (из teacher_disciplines).
2) Может выбрать дисциплину → группу → студента → выставить/изменить оценку.
3) Если поле оценки оставить пустым и нажать «Сохранить» — активная оценка «удаляется» (мягко: deleted_at = NOW()).
4) Может смотреть все оценки выбранного студента и все оценки всех студентов выбранной группы.

teacher.php (упрощённая, но рабочая версия):

```
php
<?php
require 'functions.php';
require 'db.php';
require_role('teacher');

// Находим teacher_id по user_id
$stmt = $pdo->prepare("SELECT id, full_name FROM teachers WHERE user_id = ?");
$stmt->execute([$_SESSION['user_id']]);
$teacher = $stmt->fetch();
if (!$teacher) { die('Преподаватель не найден'); }

// Дисциплины преподавателя
$stmt = $pdo->prepare("
    SELECT d.id, d.name
    FROM disciplines d
    JOIN teacher_disciplines td ON td.discipline_id = d.id
    WHERE td.teacher_id = ?
```

```
    ORDER BY d.name
");
$stmt->execute([$teacher['id']]);
$disciplines = $stmt->fetchAll();

// Группы (для фильтра)
$groups = $pdo->query("SELECT id, name FROM groups ORDER BY name")->fetchAll();
?>
<!DOCTYPE html>
<html lang="ru">
<head><meta charset="utf-8"><title>Кабинет преподавателя</title></head>
<body>
<h1>Кабинет преподавателя: <?= e($teacher['full_name']) ?></h1>
<p><a href="logout.php">Выйти</a></p>

<h2>Мои дисциплины</h2>
<ul>
<?php foreach ($disciplines as $d): ?>
  <li><?= e($d['name']) ?></li>
<?php endforeach; ?>
</ul>

<h2>Выставить / изменить / удалить оценку</h2>
<form method="post" action="save_grade.php">
  <label>Дисциплина:
    <select name="discipline_id" required>
      <?php foreach ($disciplines as $d): ?>
```

```
                <option value="<?= $d['id'] ?>"><?= e($d['name']) ?></option>
            <?php endforeach; ?>
        </select>
    </label><br><br>
    <label>Группа:
        <select name="group_id" id="group_id" required>
            <?php foreach ($groups as $g): ?>
                <option value="<?= $g['id'] ?>"><?= e($g['name']) ?></option>
            <?php endforeach; ?>
        </select>
    </label><br><br>
    <label>Студент:
        <select name="student_id" id="student_id" required>
            <!-- заполним через JS ниже, либо просто выведем всех
студентов -->
            <?php
            $allStudents = $pdo->query("SELECT id, full_name FROM students
ORDER BY full_name")->fetchAll();
            foreach ($allStudents as $s): ?>
                <option value="<?= $s['id'] ?>"><?= e($s['full_name'])
?></option>
            <?php endforeach; ?>
        </select>
    </label><br><br>
    <label>Оценка (2–5, пусто = удалить):
        <input type="number" name="grade" min="2" max="5">
    </label><br><br>
    <button type="submit">Сохранить</button>
</form>
```

```
        <h2>Просмотр оценок</h2>
        <form method="get" action="teacher.php">
            <label>Студент (все оценки):
                <select name="view_student_id">
                    <option value="">— выберите —</option>
                    <?php foreach ($allStudents as $s): ?>
                        <option value="<?= $s['id'] ?>" <?= (($_GET['view_student_id']
?? '') == $s['id'] ? 'selected' : '') ?>>
                            <?= e($s['full_name']) ?>
                        </option>
                    <?php endforeach; ?>
                </select>
            </label>
            <button type="submit">Показать оценки студента</button>
        </form>

        <form method="get" action="teacher.php">
            <label>Группа (оценки всех студентов):
                <select name="view_group_id">
                    <option value="">— выберите —</option>
                    <?php foreach ($groups as $g): ?>
                        <option value="<?= $g['id'] ?>" <?= (($_GET['view_group_id'] ??
'') == $g['id'] ? 'selected' : '') ?>>
                            <?= e($g['name']) ?>
                        </option>
                    <?php endforeach; ?>
                </select>
            </label>
            <button type="submit">Показать оценки группы</button>
```

```
</form>


<?php
// Вывод оценок выбранного студента
if (!empty($_GET['view_student_id'])) {
    $stmt = $pdo->prepare("
        SELECT g.grade, d.name AS discipline, gr.name AS group_name,
s.full_name
        FROM grades g
        JOIN students s ON s.id = g.student_id
        JOIN groups gr ON gr.id = s.group_id
        JOIN disciplines d ON d.id = g.discipline_id
        WHERE g.student_id = ? AND g.deleted_at IS NULL
        ORDER BY d.name
    ");
    $stmt->execute([$_GET['view_student_id']]);
    $rows = $stmt->fetchAll();
    echo "<h3>Оценки студента</h3>";
    if (!$rows) {
        echo "<p>Оценок нет.</p>";
    } else {
        echo "<table border='1'
cellpadding='5'><tr><th>Студент</th><th>Группа</th><th>Дисциплина</th><t
h>Оценка</th></tr>";
        foreach ($rows as $r) {
            echo "<tr><td>" . e($r['full_name']) . "</td><td>" .
e($r['group_name']) . "</td><td>" . e($r['discipline']) . "</td><td>" . e($r['grade']) .
"</td></tr>";
        }
        echo "</table>";
```

```
    }
}

// Вывод оценок всех студентов выбранной группы
if (!empty($_GET['view_group_id'])) {
    $stmt = $pdo->prepare("
        SELECT g.grade, d.name AS discipline, s.full_name
        FROM grades g
        JOIN students s ON s.id = g.student_id
        JOIN disciplines d ON d.id = g.discipline_id
        WHERE s.group_id = ? AND g.deleted_at IS NULL
        ORDER BY s.full_name, d.name
    ");
    $stmt->execute([$_GET['view_group_id']]);
    $rows = $stmt->fetchAll();
    echo "<h3>Оценки группы</h3>";
    if (!$rows) {
        echo "<p>Оценок нет.</p>";
    } else {
        echo "<table border='1'
cellpadding='5'><tr><th>Студент</th><th>Дисциплина</th><th>Оценка</th><
/tr>";
        foreach ($rows as $r) {
            echo "<tr><td>" . e($r['full_name']) . "</td><td>" . e($r['discipline'])
. "</td><td>" . e($r['grade']) . "</td></tr>";
        }
        echo "</table>";
    }
}
?>
```

```
</body>
</html>
```

save_grade.php — обработка сохранения/изменения/удаления:

```
php
<?php
require 'functions.php';
require 'db.php';
require_role('teacher');

// teacher_id по user_id
$stmt = $pdo->prepare("SELECT id FROM teachers WHERE user_id = ?");
$stmt->execute([$_SESSION['user_id']]);
$teacher = $stmt->fetch();
if (!$teacher) { die('Преподаватель не найден'); }

$student_id = (int)($_POST['student_id'] ?? 0);
$discipline_id = (int)($_POST['discipline_id'] ?? 0);
$grade_raw = $_POST['grade'] ?? '';
$grade = ($grade_raw === '') ? null : (int)$grade_raw;

if (!$student_id || !$discipline_id) {
    die('Не выбраны студент или дисциплина');
}

// Проверяем, что преподаватель действительно ведёт эту дисциплину
$stmt = $pdo->prepare("
    SELECT 1 FROM teacher_disciplines
    WHERE teacher_id = ? AND discipline_id = ?
```

```
        ");
        $stmt->execute([$teacher['id'], $discipline_id]);
        if (!$stmt->fetch()) {
            die('Вы не ведёте эту дисциплину');
        }

        // Ищем активную оценку
        $stmt = $pdo->prepare("
            SELECT id FROM grades
            WHERE student_id = ? AND discipline_id = ? AND teacher_id = ? AND deleted_at IS NULL
        ");
        $stmt->execute([$student_id, $discipline_id, $teacher['id']]);
        $existing = $stmt->fetch();

        if ($grade === null) {
            // Пустое значение = мягкое удаление активной оценки
            if ($existing) {
                $stmt = $pdo->prepare("UPDATE grades SET deleted_at = NOW() WHERE id = ?");
                $stmt->execute([$existing['id']]);
            }
        } else {
            // Валидация диапазона
            if ($grade < 2 || $grade > 5) {
                die('Оценка должна быть от 2 до 5');
            }
            if ($existing) {
                // Изменение существующей активной оценки
```

```
            $stmt = $pdo->prepare("UPDATE grades SET grade = ? WHERE id =
?");
            $stmt->execute([$grade, $existing['id']]);
        } else {
            // Новая оценка
            $stmt = $pdo->prepare("
                INSERT INTO grades (student_id, discipline_id, teacher_id, grade)
                VALUES (?, ?, ?, ?)
            ");
            $stmt->execute([$student_id, $discipline_id, $teacher['id'], $grade]);
        }
    }

    header('Location: teacher.php');
    exit;
```

Шаг 9. Функционал студента (student.php)

Студент видит только свои активные оценки.

```
    php
    <?php
    require 'functions.php';
    require 'db.php';
    require_role('student');

    // student_id по user_id
    $stmt = $pdo->prepare("SELECT id, full_name FROM students WHERE
user_id = ?");
```

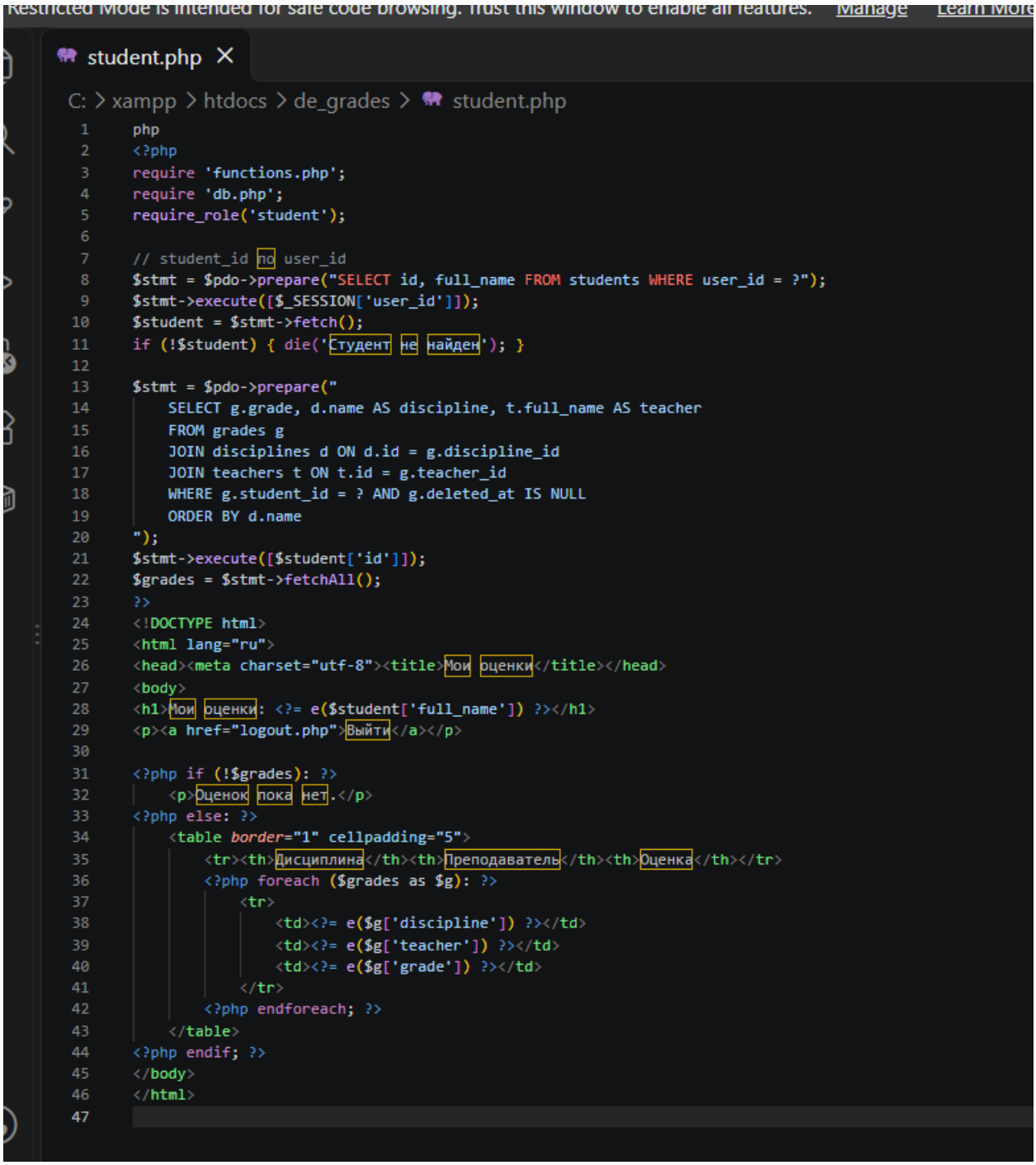
```
$stmt->execute([$_SESSION['user_id']]);
$student = $stmt->fetch();
if (!$student) { die('Студент не найден'); }

$stmt = $pdo->prepare("
    SELECT g.grade, d.name AS discipline, t.full_name AS teacher
    FROM grades g
    JOIN disciplines d ON d.id = g.discipline_id
    JOIN teachers t ON t.id = g.teacher_id
    WHERE g.student_id = ? AND g.deleted_at IS NULL
    ORDER BY d.name
");
$stmt->execute([$student['id']]);
$grades = $stmt->fetchAll();
?>
<!DOCTYPE html>
<html lang="ru">
<head><meta charset="utf-8"><title>Мои оценки</title></head>
<body>
<h1>Мои оценки: <?= e($student['full_name']) ?></h1>
<p><a href="logout.php">Выйти</a></p>

<?php if (!$grades): ?>
    <p>Оценок пока нет.</p>
<?php else: ?>
    <table border="1" cellpadding="5">

<tr><th>Дисциплина</th><th>Преподаватель</th><th>Оценка</th></tr>
        <?php foreach ($grades as $g): ?>
```

```
            <tr>
                <td><?= e($g['discipline']) ?></td>
                <td><?= e($g['teacher']) ?></td>
                <td><?= e($g['grade']) ?></td>
            </tr>
        <?php endforeach; ?>
    </table>
<?php endif; ?>
</body>
</html>
```



Шаг 10. Проверка мягкого удаления

1) Войдите как преподаватель.

2) Выставьте оценку студенту.

3) Убедитесь, что она появилась в таблице grades с deleted_at = NULL.

4) В форме выставления оценки оставьте поле пустым и нажмите «Сохранить».

5) В phpMyAdmin проверьте: у этой записи deleted_at заполнен датой, но сама запись не исчезла.

6) Убедитесь, что в интерфейсе преподавателя и студента эта оценка больше не отображается.

ПОДРОБНЕЕ НИЖЕ:

1. Войдите как преподаватель

1) Откройте браузер и перейдите по адресу: http://localhost/de_grades/login.php (если вы меняли порт, то http://localhost:8080/de_grades/login.php).

2) Введите логин: teacher1 и пароль: 12345.

3) Нажмите «Войти». Вы попадете в кабинет преподавателя (teacher.php).

2. Выставьте оценку студенту

4) На странице преподавателя найдите форму «Выставить / изменить / удалить оценку».

1) Выберите дисциплину: Базы данных.

2) Выберите группу: ИС-23-1 (или любую другую).

3) Выберите студента (например, Сидоров Алексей Петрович).

4) В поле «Оценка (2–5, пусто = удалить)» введите 5.

5) Нажмите кнопку «Сохранить».

3. Убедитесь, что оценка появилась в БД

1) Перейдите в phpMyAdmin: http://localhost/phpmyadmin.

2) Слева выберите вашу базу данных de_base.

3) Нажмите на таблицу grades (в левом меню или в центре).

4) Вы увидите новую запись. Обратите внимание на столбец deleted_at — там должно быть написано NULL (это значит, что запись активна).

4. Выполните мягкое удаление

1) Вернитесь на вкладку с сайтом (teacher.php).

2) В той же форме выберите ту же самую дисциплину, ту же группу и того же студента, что и в шаге 2.

3) Поле «Оценка» оставьте абсолютно пустым.

4) Нажмите кнопку «Сохранить».

6. Проверьте результат в phpMyAdmin

5) Снова перейдите в phpMyAdmin и откройте таблицу grades.

6) Нажмите кнопку «Обновить» (или просто обновите страницу браузера), чтобы увидеть актуальные данные.

7) Найдите ту же запись. Вы увидите, что оценка (5) осталась на месте, но в столбце deleted_at теперь стоит дата и время (например, 2023-10-27 15:30:00).

8) Важно: запись физически не исчезла из базы!

6. Убедитесь, что оценка пропала из интерфейса

Для преподавателя: На странице teacher.php в блоке «Просмотр оценок» выберите этого студента и нажмите «Показать оценки студента». Удаленной оценки там уже не будет (потому что в SQL-запросе стоит условие g.deleted_at IS NULL).

Для студента: Нажмите «Выйти» (logout). Войдите снова под логином студента (например, student1 и пароль 12345). На странице student.php этой оценки тоже не будет.

Что делать, если что-то не работает?

1) Если при сохранении пустого поля выскакивает ошибка — проверьте код в save_grade.php (страница 23-25 методички). Убедитесь, что там есть логика: если оценка пустая, выполняется UPDATE grades SET deleted_at = NOW() ....

2) Если оценка не пропадает из таблицы у студента — проверьте, что в student.php и teacher.php в запросах SELECT обязательно написано AND g.deleted_at IS NULL.

3) Если в базе оценка не меняется, убедитесь, что вы выбираете того же студента и ту же дисциплину. Если выбрать другого студента и оставить поле пустым, система просто ничего не сделает или выдаст ошибку.